



Assignment 2

Text Processing and Classification using Apache Spark

Data-Intensive Computing

Tobias Ponesch (11818774)
Jakob Olsacher (01520348)
Alana Hoogmoed (11942398)
Ekene Edeh (12451120)
Paul Hübner (00371292)

May 19, 2026

1 Introduction

The rapid growth of digital platforms has led to an unprecedented increase in the amount of textual data generated through online reviews, social media, and customer feedback, creating a need for scalable data processing and analysis solutions. Efficiently extracting meaningful information from large-scale text corpora requires distributed computing frameworks capable of handling both high computational complexity and large data volumes.

This report documents the design, implementation, and evaluation of a distributed text processing and classification pipeline using Apache Spark ¹. Building on Assignment 1, the project uses the Amazon Review Dataset 2014² within a distributed cluster environment to implement scalable preprocessing, feature extraction, and machine learning workflows.

The central objective is to construct an efficient Spark-based pipeline for transforming raw review data into machine-readable feature representations and performing automated product category classification. The assignment combines RDD-based processing, Spark ML pipeline, and Support Vector Machine (SVM) classification, while emphasizing scalability, runtime efficiency, reproducibility, and effective machine learning experiment design.

2 Problem Overview

The main challenge addressed in this assignment is the efficient processing and classification of large-scale, unstructured textual data. Since raw textual data cannot be directly processed by machine learning algorithms, the data must first be transformed into structured numerical representations. At the same time, the high dimensionality and volume of data creates computational challenges that require scalable distributed processing techniques to ensure efficient execution. An additional challenge lies in critically evaluating the implemented pipeline against the MapReduce baseline established in Assignment 1, requiring not only correct and efficient results but also meaningful interpretation of any differences observed across the two frameworks.

To address this problem, a multi-stage pipeline is implemented using Apache Spark. The approach consists of three main stages. First, Spark RDDs are used to perform text preprocessing and chi-square-based feature selection in a distributed manner. Second, the text processing workflow is reimplemented using Spark DataFrames and Spark ML pipeline components to generate TF-IDF-weighted feature vectors and select the most informative features globally. Finally, the extracted features are used in a Spark ML classification pipeline to train, tune, and evaluate a multiclass Support Vector Machine classifier for predicting product categories from review texts.

3 Methodology and Approach

This section describes the full design and implementation of the project. First, an overview of the pipeline structure is provided, followed by a detailed description of each workflow stage.

Figure 1 provides an overview of the complete project pipeline. It illustrates the data flow from the input dataset through the individual processing stages to the final output files. The final pipeline combines Spark DataFrame-based preprocessing stages with a Spark ML classification workflow, including model selection for hyperparameter optimization. The individual pipeline stages are described in more detail in the following subsections.

¹<https://spark.apache.org/>

²<https://cseweb.ucsd.edu/~jmcauley/datasets/amazon/links.html>

Assignment 2 Methodology: Spark Text Processing & Classification Pipeline

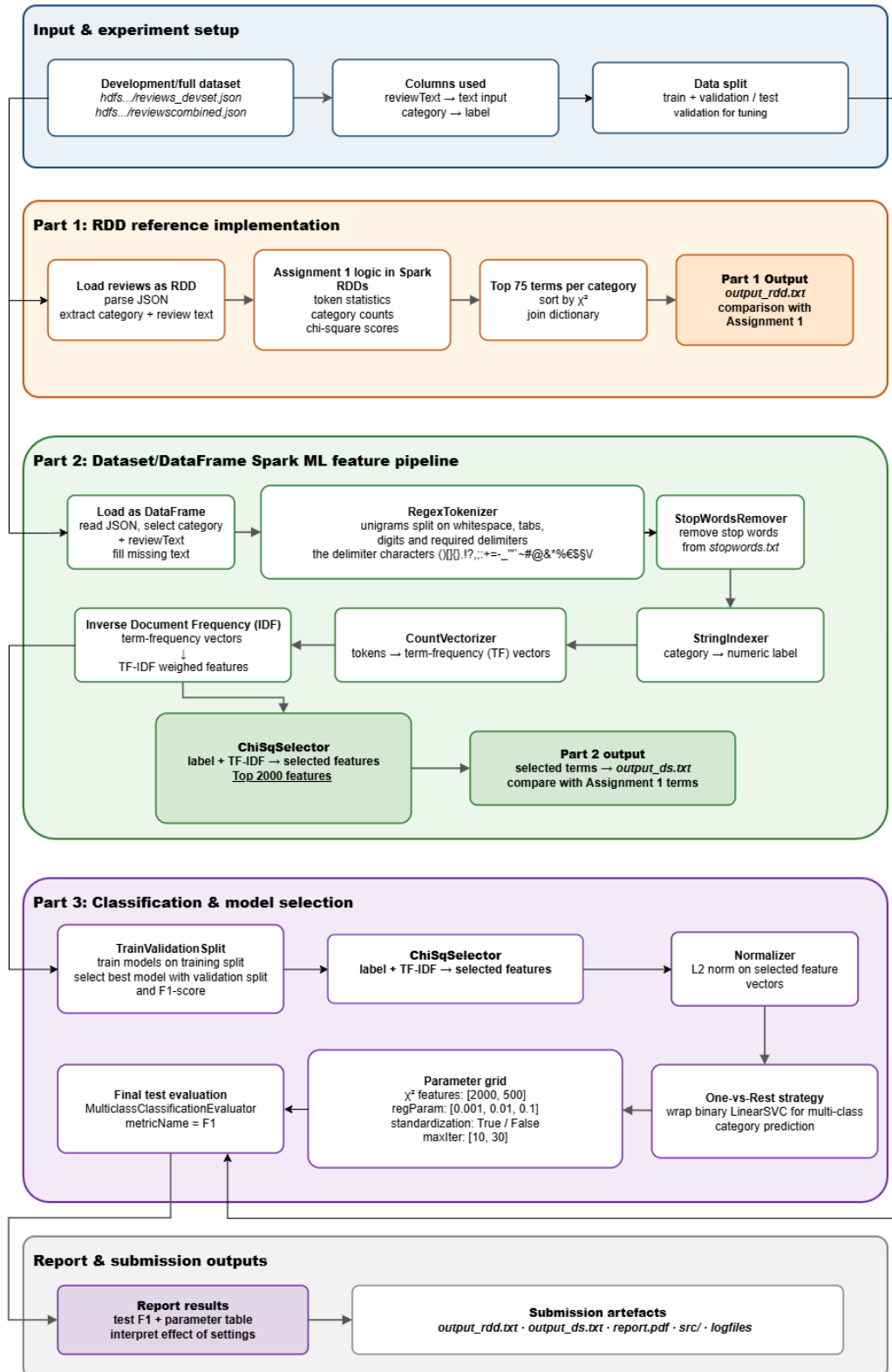


Figure 1: Overview of the implemented Spark processing and classification pipeline

3.1 Part 1: RDD-based Text Processing

During the first stage of the project, the text processing pipeline from Assignment 1 was re-implemented using Apache Spark RDDs and distributed transformations. The goal was to process the Amazon Review dataset in a scalable manner and identify category-specific terms using chi-square feature selection.

The pipeline first loaded the review dataset from HDFS and parsed each JSON record into its product category and review text. The review texts were then preprocessed through lowercasing, regex-based tokenization, and stopwords removal. Only unique words per review were retained to ensure that feature counts reflected document occurrence rather than repeated word frequency within a single review.

Using RDD transformations such as `map`, `flatMap` and `reduceByKey`, the implementation computed the following metrics:

- the total number of reviews,
- the number of reviews per category,
- the number of reviews containing each word, and
- the number of reviews in each category containing a specific word.

Based on these counts, chi-square scores were calculated for every category-word pair to measure how strongly a term was associated with a specific product category. The terms were then sorted by descending chi-square scores, and the top 75 terms per category were selected. Finally, all selected terms were merged into a global dictionary.

Note: Assignment 1 focused on low-level distributed text processing using the Hadoop MapReduce paradigm, where the data flow between mapper and reducer stages had to be explicitly designed. In contrast, this assignment re-implemented the same chi-square feature selection task using Spark RDDs and distributed transformations. This enabled a more compact and flexible implementation while leveraging Spark’s in-memory processing and higher-level distributed abstractions.

3.2 Part 2: Dataframe-based TF-IDF Feature Extraction

In a subsequent stage, the text processing workflow was re-implemented using Spark Dataframes and the Spark ML pipeline API instead of low-level RDD transformations. The objective was to transform the Amazon review texts into TF-IDF weighted vector space representation and select the most relevant features using chi-square feature selection.

The pipeline first loaded the review dataset from HDFS into a Spark Dataframe and selected the attributes `category` and `reviewText`. Missing review texts were replaced with empty strings to ensure consistent preprocessing.

The review texts were then processed through several Spark ML transformation stages including tokenization, stopwords removal, category encoding, term-frequency vector generations, as well as TF-IDF weighting.

After generating TF-IDF feature vectors, the pipeline applied `ChiSqSelector` to select the 2000 most informative terms overall based on chi-square statistics. This reduced the dimensionality of the feature space while preserving the terms most strongly associated with the product categories.

All preprocessing and feature engineering stages were combined into a single Spark ML Pipeline, which allowed the transformations to be executed sequentially in a scalable and reproducible way. After fitting the pipeline to the dataset, the selected features indices were mapped back to their corresponding vocabulary terms.

Compared to the first stage, this approach relied entirely on Spark ML’s built-in dataframe-based transformations rather than manually implemented RDD operations. This simplified the preprocessing workflow, improved integration with later machine learning tasks, and prepared the feature extraction pipeline for the subsequent classification.

3.3 Part 3: Multiclass Text Classification

In the final stage of the project, a multiclass text classification pipeline was implemented using Spark ML and Support Vector Machines. The objective was to classify Amazon reviews into their corresponding product categories based on the textual review content.

As in previous stages, the pipeline first loaded the review dataset from HDFS, selected the attributes category and reviewText, replaced missing review texts with empty strings, and lastly repartitioned the dataset to improve parallelism during distributed execution. The dataset was then split into training/validation data and a separate test set using a fixed random seed to ensure reproducibility.

The preprocessing and feature engineering stages closely followed the workflow established in the previous stage. The review texts were tokenized, cleaned through stopword removal, transformed into term-frequency vectors, and weighted with TF-IDF scores. However, in contrast to Part 2, the pipeline was split into two separate parts for the classification task. First, a preprocessing pipeline consisting of tokenization, stopword removal, label indexing, term-frequency vectorization, and IDF weighting was fitted only on the training/validation split. The fitted preprocessing model was then used to transform both the training/validation data and the test data. This ensured that the vocabulary, label mapping, and IDF statistics were learned without using information from the test set.

This separation also improved computational efficiency during hyperparameter tuning. Since the preprocessing output was cached as precomputed TF-IDF features, expensive steps such as tokenization, vocabulary construction, and IDF weighting did not have to be repeated for every grid-search configuration. Instead, the subsequent model-selection pipeline only contained the stages affected by the parameter grid: chi-square feature selection, L2 normalization, and the classifier.

After preprocessing, chi-square feature selection was applied to retain the most informative features for classification. The `ChiSqSelector` was kept inside the model-selection pipeline because the number of selected features was part of the hyperparameter grid. Therefore, feature selection was performed during model selection and evaluated together with the classifier parameters. The selected feature vectors were then normalized using L2 normalization before being passed to the classifier.

For the classification task, the project used Spark ML’s `LinearSVC` implementation combined with the `OneVsRest` strategy to support multiclass classification. Since linear SVMs are binary classifiers, the One-vs-Rest approach trained one classifier per category and selected the category with the highest prediction score for each review.

To optimize the classification performance, a hyperparameter grid search was implemented. The experiments evaluated multiple combinations of:

- number of selected chi-square features: 500 and 2000,
- SVM regularization parameter `regParam`: 0.001, 0.01, and 0.1,
- feature `standardization`: `True` and `False`
- maximum number of training iterations `maxIter`: 10 and 30.

Model performance was evaluated using the multiclass F1-score. After training all parameter combinations, the best-performing configuration on the validation data was selected and evaluated on the test dataset. The final output file contained all tested parameter combinations, validation scores, the best hyperparameter configurations, as well as the final test F1-score.

4 Results

4.1 Comparison of Assignment 1 and Part 1

The comparison between the output of Assignment 1 and the output of Assignment 2 Part 1 shows that both implementations produce almost identical results. The main difference between the two versions is the tokenization strategy. In Assignment 1, the regular expression `re.compile(r"[\p{L}\p{Z}<>|\"]+")` was used. This expression defines which characters are allowed inside a token: uppercase and lowercase letters as well as the special characters `<`, `>`, `^` and `|`. Every character outside this allowed set is treated as a delimiter. Therefore, this approach is relatively restrictive, because tokens can only consist of these explicitly allowed characters.

In Assignment 2 Part 1, the regular expression was changed to directly represent the delimiters specified in the assignment. Instead of defining allowed token characters, this expression lists the characters that should be used to split the text, such as whitespace, digits, brackets, punctuation marks, quotation marks, currency symbols and slashes. This makes the tokenization more closely aligned with the assignment specification.

Despite this change, the comparison shows that the resulting categories are the same in both outputs. Furthermore, for almost all categories, the selected terms are also identical. The only category where a difference occurs is `Tools_and_Home_Improvement`. In Assignment 1, the term `kidde` appears in this category, whereas in Part 1 the term `bidet` appears instead. Both terms have exactly the same Chi-Squared value of 159.72357729489363. Therefore, this difference is not caused by a computational error, but by a tie in the Chi-Squared ranking. Since only a fixed number of top terms is retained, one of the tied terms can be included while the other one is cut off, depending on the ordering.

Apart from this single mismatch, all overlapping terms have identical Chi-Squared values in both outputs. The overall term list also differs only by the same two terms, namely `kidde` and `bidet`. This confirms that the implementation of Part 1 is consistent with Assignment 1. The observed difference is only the result of a tie-breaking situation at the selection boundary, while the actual Chi-Squared computations match.

In addition to producing equivalent results, the Spark RDD implementation was faster on the development dataset. The RDD version completed in 72 seconds on the cluster, while the Hadoop implementation required 220 seconds. This corresponds to a runtime reduction of approximately 67%. The improvement can be attributed to Spark's in-memory processing model and the more compact execution of repeated distributed transformations compared to the disk-based MapReduce workflow.

4.2 Comparison of Assignment 1 and Part 2 of Assignment 2

Part 2 differs more strongly from Assignment 1 than Part 1 does, because it uses a Spark ML-based feature engineering and selection pipeline instead of the manual category-wise chi-square procedure. Assignment 1 selects the top 75 terms separately for each category and then merges these lists, which favors terms that are highly characteristic for individual classes. In contrast, Part 2 applies `RegexTokenizer`, `StopWordsRemover`, `StringIndexer`, `CountVectorizer`, `IDF`, and `ChiSqSelector`, selecting features globally with respect to the multiclass category label.

The notebook results reflect this difference. Assignment 1 contains 1464 unique selected terms after merging the category-specific top lists, while Part 2 contains 2000 selected terms. The maximum possible number for Assignment 1 would be $22 \times 75 = 1650$, but the actual number is lower because some terms appear in the top lists of multiple categories. The overlap between Assignment 1 and Part 2 consists of 751 terms. Since Assignment 1 and Part 1 differ only by one tied term, the differences between Part 1 and Part 2 can mainly be attributed to the changed feature selection strategy rather than to an implementation error.

The feature representation also changes in Part 2. Assignment 1 and Part 1 use review-level term occurrences for the manual chi-square calculation, whereas Part 2 builds term-frequency vectors with `CountVectorizer` and applies IDF weighting before feature selection. As a result, Part 2 takes into account both how often a term occurs in a review and how informative it is across the corpus, not only whether it appears.

Overall, Assignment 1 and Part 2 are not expected to produce identical term lists. Their partial overlap shows that both approaches identify some common important terms, while the remaining differences are mainly due to Part 2’s global Spark ML feature selection, TF-IDF representation, and different tokenization strategy.

In terms of runtime, Part 2 required 138 seconds on the development dataset in cluster mode, while the RDD-based Part 1 implementation completed in 72 seconds. Thus, Part 2 was slower than Part 1 for this dataset. This is expected because Part 2 performs additional Spark ML feature engineering steps, including vocabulary construction with `CountVectorizer`, IDF weighting, and global chi-square feature selection over 2000 features. However, the output of Part 2 is also more directly suited for the subsequent machine learning pipeline, since it produces a TF-IDF-based feature representation rather than only a category-specific term dictionary.

4.3 Results of Part 3: Multiclass Text Classification

The multiclass SVM experiment was executed on the development dataset with 78,829 reviews. The data was split reproducibly into 66,993 training/validation instances and 11,836 test instances. Model selection was performed using a grid search over 24 parameter combinations, varying the number of selected chi-square features, the SVM regularization parameter, feature standardization, and the maximum number of iterations.

Features	regParam	Standardization	maxIter	Validation F1
2000	0.001	True	10	0.6088
2000	0.001	True	30	0.6083
2000	0.001	False	10	0.5154
2000	0.001	False	30	0.5429
2000	0.010	True	10	0.6095
2000	0.010	True	30	0.6082
2000	0.100	True	10	0.6068
2000	0.100	True	30	0.6000
500	0.001	True	10	0.4749
500	0.010	True	10	0.4765
500	0.100	True	10	0.4674
500	0.100	False	10	0.0020

Table 1: Representative validation F1 scores from the Part 3 SVM grid search. The table shows a compact subset of the evaluated configurations, including the best-performing setting and selected lower-performing configurations for comparison. The complete results for all 24 parameter configurations are reported in `output_svm.txt`.

The best validation result was achieved with 2000 selected chi-square features, `regParam=0.01`, enabled standardization, and `maxIter=10`. This configuration reached a validation F1-score of 0.6095 and a final test F1-score of 0.6061.

The results show that the 2000-feature representation clearly outperformed the stronger 500-feature filtering. The best 500-feature configuration reached only 0.4765 validation F1, which indicates that the smaller feature set removed too much relevant information for the multiclass classification task. Standardization had a consistently positive effect, especially for the 2000-feature configurations. Increasing the maximum number of iterations from 10 to 30 did not improve performance, suggesting that the model already converged sufficiently for the best-performing settings. The close match between validation F1 and test F1

indicates that the selected model generalized consistently to unseen data. The total runtime for the grid-search was ≈ 4.9 hours.

5 Conclusion

5.1 Design Choices and Efficiency Considerations

The results show that the chosen pipeline structure provides a reasonable trade-off between manual control and higher-level Spark abstractions. The RDD-based implementation was suitable for reproducing the category-wise chi-square logic from Assignment 1, because it allowed explicit control over distributed counting and aggregation. In contrast, the Spark ML-based implementations were more appropriate for feature extraction and classification, since they provided reusable pipeline stages and better integration with machine learning components.

Efficiency was mainly limited by the high dimensionality of the text features and by the computational cost of the multiclass SVM classifier. Therefore, feature selection played an important role in reducing the feature space before classification. The results also showed that using a larger selected feature set of 2000 features led to better classification performance than using only 500 features, indicating that too aggressive dimensionality reduction removed relevant information.

5.2 Cluster Performance Issues and Optimizations

The main performance bottleneck occurred in Part 3 during grid search for the multiclass SVM classifier. In the initial implementation, the full preprocessing and classification pipeline was recomputed for every hyperparameter configuration. Together with the default cluster configuration, this caused the first run to continue for approximately 13 hours until the run was interrupted.

To reduce runtime, the pipeline was split into a preprocessing pipeline and a model-selection pipeline. The preprocessing stages up to IDF weighting were fitted once on the training/validation data and cached as TF-IDF features. The grid search then only recomputed the stages affected by the parameter grid, namely `ChiSqSelector`, `L2-normalization`, and the `OneVsRest` Linear SVM classifier. This avoided repeated vocabulary construction and IDF computation during model selection.

Additional optimizations were applied at the cluster and Spark configuration level. The final run used six executors with two cores each, increased Spark parallelism settings, enabled parallel execution for model selection and One-vs-Rest classification, and repartitioned the input `DataFrame` into 64 partitions before caching. With these changes, the final Part 3 run completed in 17,545 seconds, corresponding to approximately 4.9 hours.

Overall, the optimization mainly improved runtime by avoiding repeated preprocessing and by using the available cluster resources more effectively. However, the classification stage remained computationally expensive because One-vs-Rest trains one binary SVM per category and this process is repeated for multiple grid-search configurations. This makes the approach scalable, but still costly for large multiclass text classification tasks.